

For loops

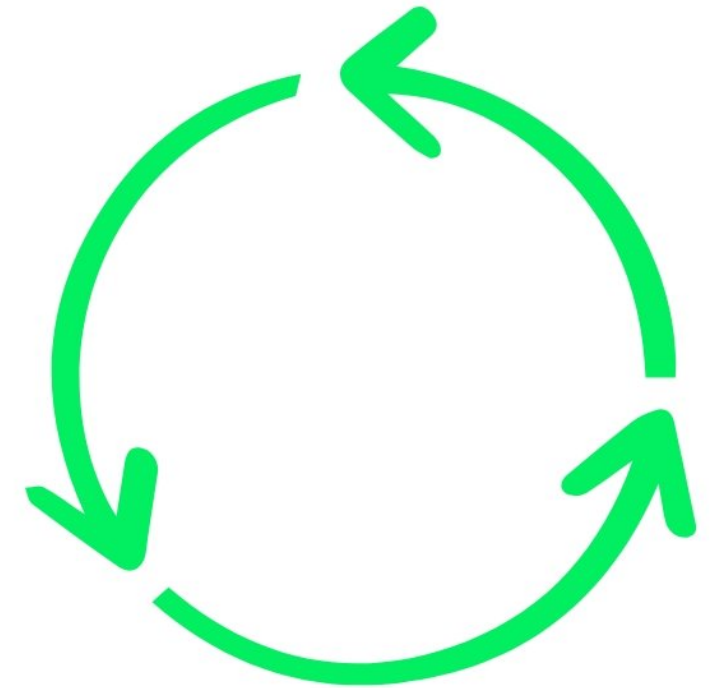
INTERMEDIATE JAVA



Jim White
Java Developer

Loops

- Let us repeat an action multiple times
- Most common in Java: `for` loop
 - Used when we know in advance how many repetitions we want



for loop syntax

```
for (int i = 0; i < 5; i++) {  
  
    // Do something  
  
}
```

- `i` : iterator
- `int i = 0` : start with `i` equal to `0`
- `i < 5` : continue while `i` is smaller than `5`
- `i++` : increase `i` by `1` after each loop
- iterable: thing we loop over
 - e.g., `String` or `array`

for loop example

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Hello".charAt(i));  
}
```

```
H  
e  
l  
l  
o
```

- `i` is iterator
- `"Hello"` is iterable

for loop through an array

```
// Iterate through array and print each element
int[] num = {1, 2, 3, 4, 5};

for (int i = 0; i < num.length; i++) {
    System.out.println(num[i]);
}
```

```
1
2
3
4
5
```

Adding to array in a loop

```
// Iterate through array
// and add i to each element
int[] num = {1, 2, 3, 4, 5};

for (int i = 0; i < num.length; i++) {
    System.out.println("Old: " + num[i]);
    num[i] += i;
    System.out.println("New: " + num[i]);
}
```

```
Old: 1
New: 1
Old: 2
New: 3
Old: 3
New: 5
Old: 4
New: 7
Old: 5
New: 9
```

Infinite loops

- Causes:
 - Not updating iterator
 - Wrong stopping condition
 - ...
- Avoid infinite loops!
 - Cause program to freeze or crash

Examples

```
// i is always bigger than 3
for (int i = 3; i > 2; i = i + 1) {
    System.out.println("Forever...");
}
```

```
// Not updating i
for (int i = 0; i < 10; i = i - 0) {
    System.out.println("And ever...");
}
```


Break

- Used to exit loop early
- Not a good coding practice, use wisely
 - `while` loops is usually better

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
    if (i == 1) {  
        break; // Breaks the loop when i is 1  
    }  
}
```

0

1

Recap

```
// Example of a for loop
// Including the break keyword
String word = "Hey";

for (int i = 0; i < word.length(); i++){
    System.out.println(word.charAt(i));
    if (i == 2) {
        System.out.println("!");
        break;
    }
    System.out.println("-");
}
```

- `i` : iterator
- `word` : iterable
- `for` loop runs for predetermined amount of loops
 - Runs from `0` till the end of `word`
 - Unless we use the `break` keyword

```
H
-
e
-
y
!
```

Let's practice!

INTERMEDIATE JAVA

Foreach loops

INTERMEDIATE JAVA



Jim White
Java Developer

foreach

- Allows iterating through things like arrays without managing indexes
- Useful when we don't need to know where the value is



foreach syntax

```
String[] names = {"Ada", "Ivo", "Ti"}

for (String name: names){
    System.out.println(name);
}
```

Ada

Ivo

Ti

- `name` : temporary variable, needs to have a type
- `names` : array (or generally an iterable)

On each iteration, Java automatically gives us the next element of the iterable

For each name in names, do something

for each loop

Pros

- Improves readability over normal `for` loop
- Removes extra setup
- Better for:
 - Working with values as they are
 - E.g., printing the values

Cons

- Not working with indexes
- Can only use the value we get
- Bad for:
 - Changing values
 - Keeping track of position

Let's practice!

INTERMEDIATE JAVA

While loops

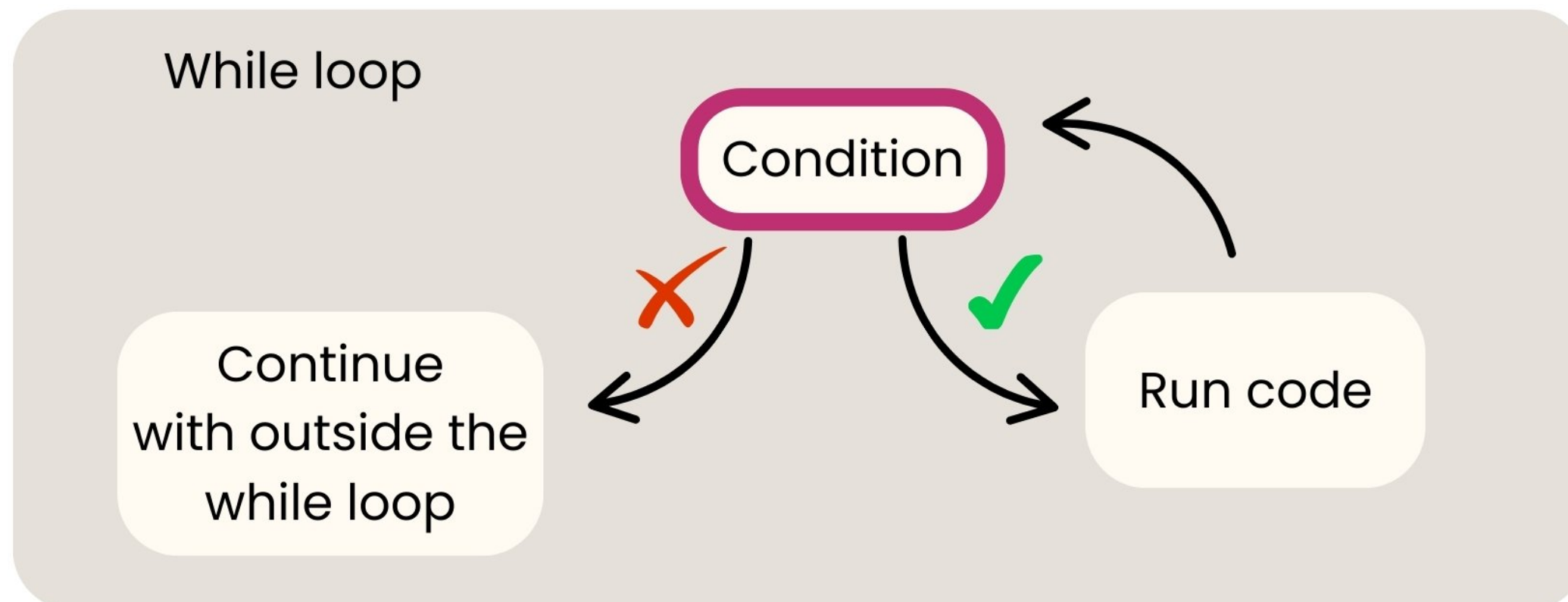
INTERMEDIATE JAVA



Jim White
Java Developer

How while loop works

- First check condition
 - If true, we run the code provided
 - If not, the loop ends
- Repeat: check condition and then decide



while loop syntax

```
while (condition) {  
  
    // Run this code  
  
}
```

- Syntax similar to `if` statement
- Unlike `if` statement, can run multiple times

while loop example

```
// Prints numbers smaller than 5
int counter = 1;

while (counter < 5){
    System.out.println(counter);
    counter = counter + 1;
}
```

```
1
2
3
4
```

Infinite while loop

- Forgot to update variable

```
int counter = 1;

while (counter < 5){
    System.out.println(counter);
    counter = counter + 0;
}
```

➔ Causes program to hang

- Condition is always `true`

```
int counter = 1;

while (counter < 5){
    System.out.println(counter);
    counter = counter - 1;
}
```

Updating operators

```
int counter = 0;
```

```
// Same as counter = counter + 3  
counter += 3;
```

Similar to `-=` , `*=` , `/=` , ...

Break

- We can use `break` to exit `while` loop early, but not best coding practice

```
int counter = 1;
while (counter < 5){
    System.out.println("Counter: " + counter);
    if (counter == 3) {
        break; // Exit the loop when counter is 3
    }
    counter+=2;
}
```

Counter: 1

Counter: 3

Recap

```
// while loop that prints even numbers smaller than 20
int counter = 2;

while (counter < 20){
    System.out.println(counter);
    counter *= 2;

    // Exit while loop if counter is odd
    if (counter % 2 == 1){ // counter % 2 is counter modulo 2
        System.out.println("Something went wrong, counter is odd.");
        break;
    }
}
```


Let's practice!
INTERMEDIATE JAVA